

# Table for Four — Project Scoping & Initial Agent Design

---

## CMU AI Agent Certification — Capstone · Week 1 Submission

Author: Manish Bhatt

This is the Week 1 scoping submission. It is a living document that will be expanded over the 6–7 week program as each milestone is built. A fuller design reference is maintained alongside it in the project repository.

## 1. Overview

---

**Table for Four** is an agentic restaurant concierge. A user makes a natural-language dining request — *"Italian, near Midtown, 4 people, Friday 7pm, one guest is gluten-free"* — and the agent **searches** real restaurant data, **reasons** over which options fit, surfaces relevant **perks** (offers/coupons), and **books** a table — with a **human-in-the-loop confirmation** step before any booking is finalized, and a full **governance/audit trail**.

The system is built **MCP-first** (each capability is a Model Context Protocol tool server) and **orchestrated with LangGraph**, so reasoning, tool use, human approval, and auditing are explicit and inspectable rather than hidden inside one prompt.

## 2. Problem statement

---

Booking a restaurant for a group is a small but real **multi-constraint decision problem**: reconciling cuisine, location, party size, timing, budget, dietary needs, ratings, and availability — then acting on it. It is retrieval-heavy (depends on current real-world data) and **transactional** (ends in an irreversible action a responsible system should not take without human confirmation). That makes it an ideal, well-bounded testbed for an agent that must **search, reason, personalize, and act under human oversight** — exercising the full agent loop.

## 3. Goals & non-goals

---

### Goals

- Turn a free-text request into a **ranked, reasoned shortlist** using **real** restaurant data.
- **Personalize** options with relevant perks via **semantic retrieval (RAG)**.
- **Book** a table, gated by **explicit human confirmation**.
- Produce a **complete audit trail** with data provenance.
- Run **fully offline with no API keys** for development/grading, and switch to **live data** when keys are provided.

### Non-goals (out of scope)

- Real transactional booking against partner-gated systems (OpenTable/Resy) — a **mock backend** is used by design.
- Payments, real coupon redemption, production UI, or identity systems.

## 4. Primary scenario (happy path)

- User: *"Italian, near Midtown, 4 people, Friday 7pm, one guest is gluten-free."*
- Agent searches and filters restaurants; returns a shortlist with reasoning.
- Agent attaches **relevant perks** ("2 of 3 have offers fitting a GF group of 4").
- User picks an option.
- Agent proposes a booking and **pauses for explicit confirmation**.
- On approval, the booking is finalized and confirmed.
- Every step is logged with its data source.

## 5. Architecture (high level)

```

User (chat / CLI)
  | natural-language request
  v
Orchestrator Agent (LangGraph state machine)
  plan -> search -> match perks -> propose -> [HUMAN GATE] -> book
    |           |           |           |
  Search MCP   Perks MCP   Booking MCP   Governance /
  (Google      (Chroma vector (mock FastAPI   Audit layer
  Places +     DB, synthetic  reservation   (tool calls,
  fixture)     perks)         backend)       sources, approvals)
  
```

### Design principles

- **MCP-first:** every capability is a discrete, swappable, independently testable tool server.
- **Offline-first, live-optional:** each path has a keyless offline mode; the same normalization code runs in both modes.
- **Provenance is first-class:** every result carries a `source` field (`live` | `fixture` | `synthetic`).
- **The irreversible step is gated:** no booking without explicit human approval.

## 6. Agent design (summary)

**Orchestration.** A LangGraph **state graph** with explicit nodes: `parse_request` → `search` → `match_perks` → `rank_and_explain` → `propose_booking` → `human_gate` (interrupt for approval) → `book`, with a cross-cutting `audit` step. State is carried explicitly so the flow is inspectable and resumable.

### Tools (MCP servers).

- **Search** — `search_restaurants` (*built*): Google Places API (New) with an offline fixture fallback; normalized results; price/rating/open-now filters; a cost-controlling field mask; returns `source`.

- **Perks** — `find_perks` (*planned*): Chroma vector DB of **synthetic** perks; **hybrid retrieval** = semantic similarity over an unstructured blurb + structured metadata filters (party size, expiry, type); local embeddings, no key.
- **Booking** — `create_booking` (*planned*): self-built **mock FastAPI** backend; availability + reservation with a confirmation id.

**Human-in-the-loop gate.** Before any booking is written, the graph interrupts and surfaces the proposed reservation for **approve/decline** — the central responsible-AI control.

**Governance/audit.** A structured entry per consequential step: the tool called, the **data source**, the **human approval**, and the booking outcome.

## 7. Data strategy

| Capability        | Source                                              | Rationale                                                                                                             |
|-------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Restaurant search | <b>Real</b> — Google Places (New) + offline fixture | Real retrieval is core; fixture keeps it keyless/testable.                                                            |
| Perks / coupons   | <b>Synthetic</b> , clearly labeled                  | No cleanly-licensable free coupon dataset; labeled synthetic data is reproducible and the stronger governance choice. |
| Booking           | <b>Self-built mock</b> (FastAPI)                    | Real reservation APIs are partner-gated; mocking a gated downstream is a standard prototyping pattern.                |

Every payload carries an explicit `source` ; synthetic/mock data is never presented as live — the labeling **is** part of the responsible-AI design.

## 8. Technology stack

Python ≥ 3.12 (via `uv`) · **MCP** (FastMCP) for tools · **LangGraph** for orchestration · **Chroma** + local `all-MiniLM-L6-v2` embeddings for perks RAG · **FastAPI** for the mock booking backend · `httpx` , `python-dotenv` , `pytest` .

## 9. Roadmap (6–7 weeks)

| #   | Milestone                                                            | State      |
|-----|----------------------------------------------------------------------|------------|
| 1   | Search MCP server (Google Places, offline-testable)                  | ✅ Complete |
| 2   | Mock booking FastAPI + booking MCP server                            | 🟡 Next     |
| 2.5 | Perks / RAG: synthetic perks → Chroma → <code>find_perks</code> tool | 🟡 Planned  |
| 3   | LangGraph orchestrator; end-to-end happy path                        | 🟡          |
| 4   | Human-in-the-loop gate + governance/audit logging                    | 🟡          |
| 5   | (Stretch) member preferences, model comparison, demo                 | 🟡          |

## 10. Responsible AI (highlights)

- **Human authority over irreversible actions** — bookings require explicit approval.
- **Data provenance & honesty** — `source` labeling on every payload.
- **Auditability** — a structured trail of tool calls, sources, and approvals.
- **Cost control & least privilege** — the Places field mask fetches only needed fields, minimizing data and billing.
- **Secret hygiene** — keys live only in a gitignored `.env`.
- **Reproducibility** — offline modes and seeded synthetic data make runs gradable without credentials.

*Week 1 submission · v1.0 — this document will grow with each milestone.*